

01211373 Machine Learning and Programming for Industry

DEEP LEARNING

Recurrent Networks & Sequential Data

Lecture 10 — Forecasting Tool Wear

PyTorch • Ch. 15

Last Week → This Week

WEEK 9 RECAP

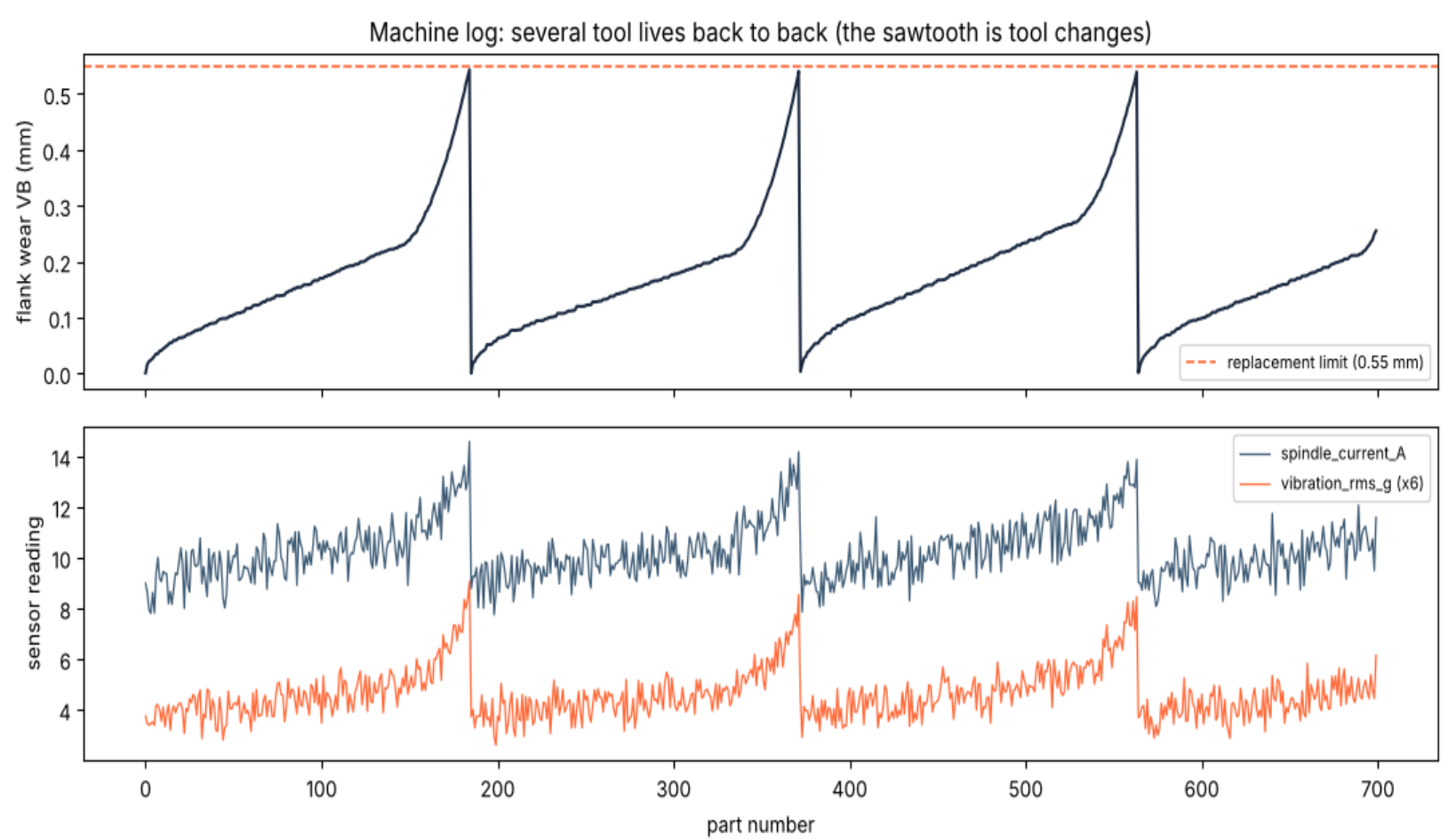
- Flattening an image destroys neighbourhood structure
- Convolution + weight sharing exploit it
- A detector learned in one place works everywhere
- Structure built into the architecture beat raw capacity

TODAY

- A new kind of structure: temporal order
- Turning a sensor log into training examples
- Recurrent networks and the hidden state
- Why plain RNNs forget, and how LSTM gates fix it
- Proving a model really uses order

Today's Problem: Tool Wear on a Machining Centre

Flank wear climbs through three phases, the insert is swapped at 0.55 mm, and wear resets. Forecast VB 20 parts ahead from four sensor channels.



1. Break-in

rapid, decelerating

2. Steady state

slow, roughly linear

3. Breakdown

rapid, accelerating

Tool change

wear resets to zero

You cannot measure VB in production. You can measure the sensors.

Order Is Information



Here are two windows containing exactly the same set of sensor values. Only the order differs — and the correct forecasts are opposite.

High, then low

The insert was just replaced. The high readings are the old worn tool; the low ones are the new one.

Forecast: wear stays LOW

Low, then high

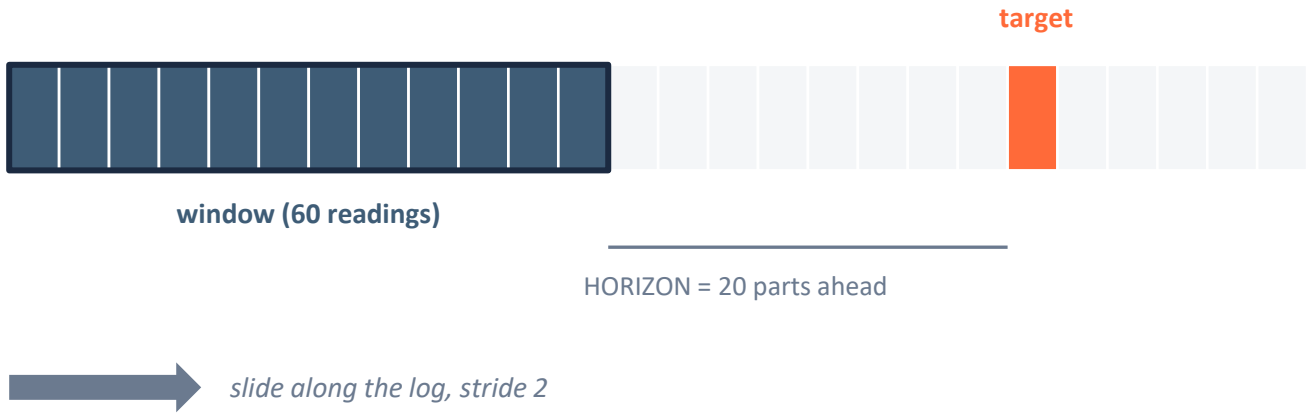
A fresh tool is wearing in and the readings are climbing.

Forecast: wear keeps RISING

Mean, standard deviation, minimum, maximum — all identical. Any model that treats the window as an unordered bag of numbers cannot tell these two situations apart. That is the gap a sequence model fills.

From a Log to Supervised Examples

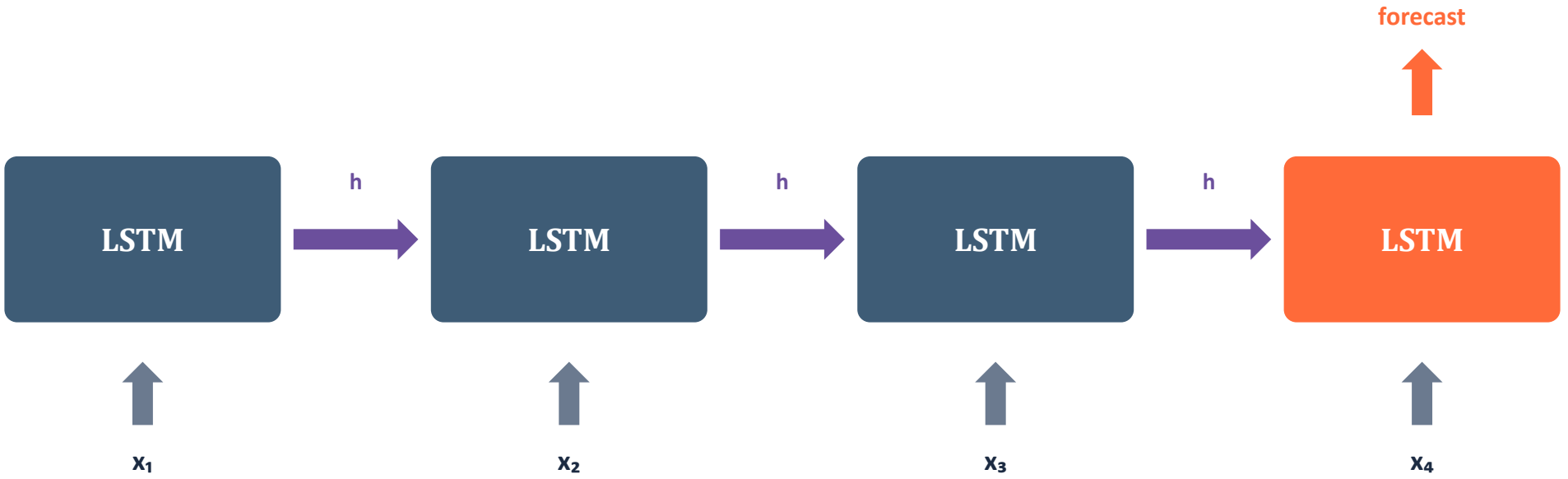
A time series is not yet a training set. Slide a window along the log: the readings inside it are the features, and a value HORIZON steps later is the target.



The time-series version of Week 6's leakage rule: neighbouring windows overlap by 58 of their 60 readings. Split them randomly and near-duplicates land on both sides — your test score measures memorization. Split by machine log, or by time.

The Recurrent Idea: Carry a Hidden State

Read the sequence one step at a time. At each step, combine the new reading with what you already remember, and pass the result forward.

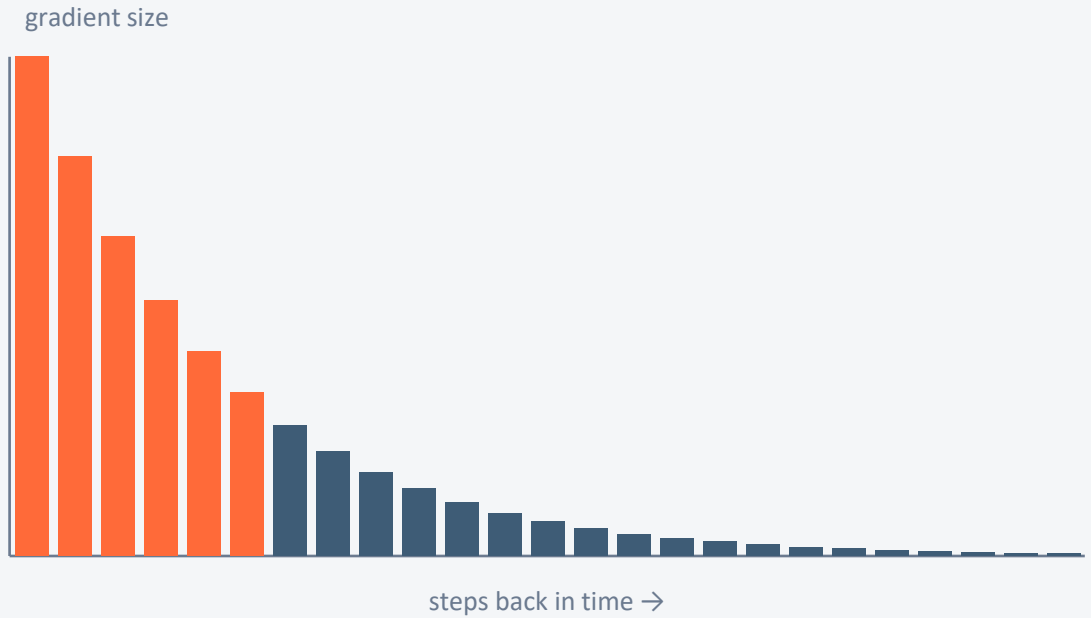


the same weights are reused at every step — weight sharing, along time instead of across space

Only the last step's output is used here. That is a many-to-one task: a whole sequence in, one number out. Forecasting the next 20 values instead would be many-to-many.

Why a Plain RNN Forgets

Training a recurrent network means multiplying gradients back through every time step. Repeated multiplication by a factor below one shrinks them toward nothing.



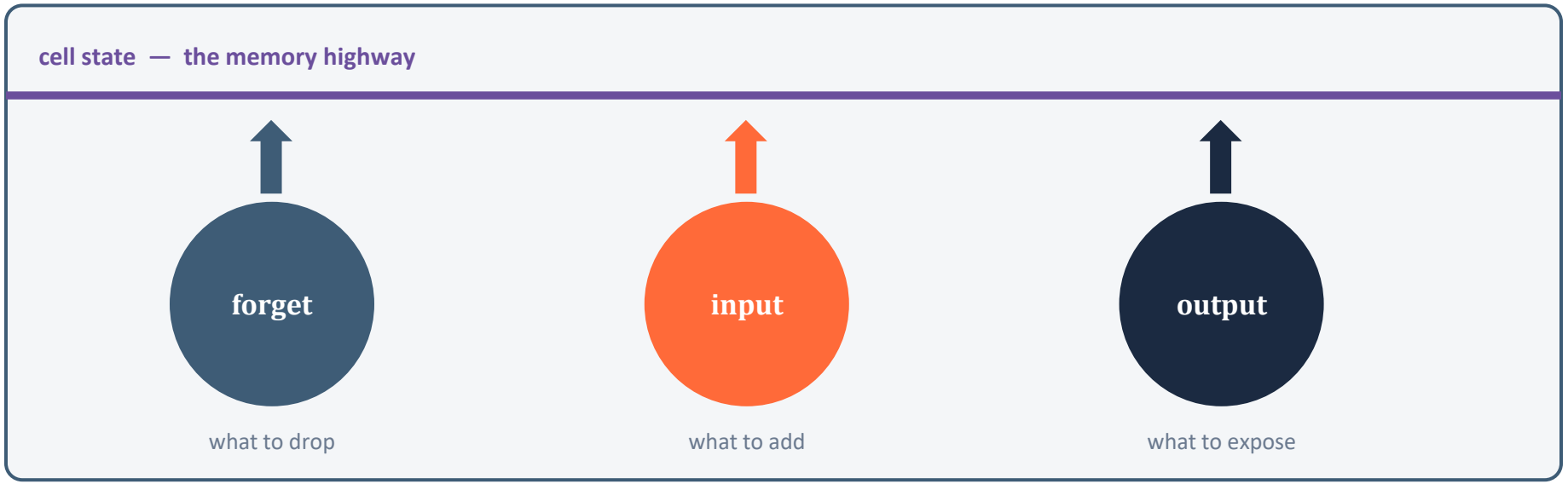
after ~20 steps the gradient is effectively zero — the network cannot learn long-range dependencies

This is the same vanishing-gradient problem that made sigmoid a poor hidden activation in Week 7 — but far worse, because the depth here is the length of the sequence.

A 60-step window is effectively a 60-layer network.

The LSTM Cell: A Memory With Gates

An LSTM adds a cell state that runs straight through, modified only by gates. Gradients can flow along it without being repeatedly squashed.



Each gate is a small learned layer that outputs a number between 0 and 1 per channel — 0 closes it, 1 opens it fully.

The key structural change: the cell state is modified by addition, not by repeated multiplication. That is what keeps gradients alive across long sequences.

Sequence Shapes in PyTorch



Week 9's bug was the missing channel dimension. The sequence equivalent is choosing the wrong time step from the output.

```
input   (batch, seq_len, n_channels)   →   output   (batch, seq_len, hidden)
```

batch_first=True

Without it PyTorch expects (seq_len, batch, features) — the classic silent shape bug.

out[:, -1, :]

One output per time step comes back. For a many-to-one forecast, take the LAST — the only one that has seen the whole window.

Check before training

Push four examples through and print the output shape. It costs a second and catches most sequence bugs.

The Baseline Worth Beating



Summarise each window by four statistics per channel — mean, standard deviation, min, max — and train an MLP on those. It is blind to order by construction.

What it can see

- The overall level of each channel
- How much the readings spread
- The extremes reached in the window

What it cannot see

- Whether readings rose or fell
- Whether the rate is speeding up or slowing
- That a tool change happened at all

This is the honest comparison. It is not a weak strawman: these four statistics are exactly what an experienced engineer would hand-pick. The question is whether a sequence model can match them without being told what to measure.

The Result

Forecast error on a held-out machine log, 1209 windows. Three training seeds each — one run is a number, three are evidence.

MLP on summary statistics

median

per-seed: 39.1 38.0 41.3 µm

39.1 µm

LSTM on ordered sequences

median

per-seed: 34.9 34.5 31.8 µm

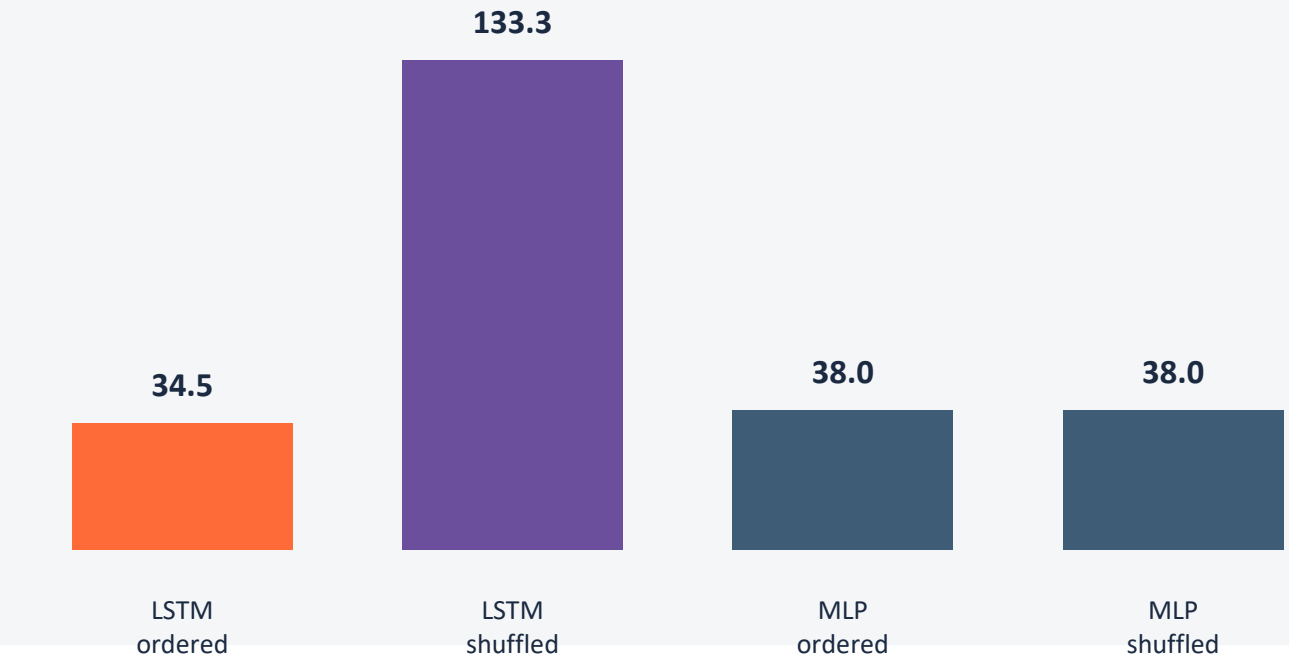
34.5 µm

The sequence model wins by about 12%, and the gap is larger than the spread between seeds — so it is a real difference, not luck.

Nobody told it to measure a mean or a slope. It built its own summary of the window from raw readings.

Proving It: Destroy the Order

Take the trained LSTM and feed it test windows whose time steps have been randomly permuted. Same values, same multiset, order gone.



Four times worse. The LSTM's accuracy depended on the order, so it had genuinely learned temporal structure.

On the windows containing a tool change — where order matters most — the collapse is sharper still: 30.5 → 250.7 µm.

The MLP does not move at all. Its features are permutation-invariant, so it never had the information to lose — a built-in control.

How Much Context? A Real Design Decision

The window length is not a detail. With only 30 readings the LSTM's advantage over summary statistics largely disappears — the lab asks you to verify this.

Short window (30)

Four statistics capture most of what 30 numbers contain. There is little left for a sequence model to exploit, and a tool change plus the trend that follows it does not fit inside.

Longer window (60)

The same four statistics now compress twice as much away. The ordered trajectory carries information the summaries discard — and the sequence model pulls ahead.

But longer is not always better. Every extra step is another layer for the gradient to travel back through, and more compute. Window length trades context against trainability — the tension Week 11 resolves differently.

Today's Lab: Forecast, Then Prove It

1	**Build windows from a machine log**	Mind the indices — and do not leak the future.
2	**Beat an order-blind baseline**	Summary statistics an engineer would pick by hand.
3	**Train an LSTM over three seeds**	Report the median, not your luckiest run.
4	**Shuffle time and watch it break**	Then turn the forecast into a maintenance decision.

Key Takeaways

- In a time series the order of the readings is itself information.
- A hidden state carries the past forward; LSTM gates keep the gradient alive.
- If shuffling your inputs does not hurt the model, it was never using order.

NEXT WEEK

Attention & Transformers

A hidden state is a fixed-size bottleneck. What replaced it, and why it reads the whole sequence at once.